

ENDPOINT SECURITY PROJECT

USB Drive Access Control via Polkit

Ubuntu Linux | PolicyKit | GDPR Art. 32 Compliant

STATUS: LIVE	GDPR ART. 32	JUNE 2026
--------------	--------------	-----------

Prepared By	Karthikeyan T
Platform	Ubuntu Linux (GNOME Desktop Environment)
Tech Stack	Polkit (PolicyKit) · udisks2 · dconf
Document Type	Technical Reference

1. Project Overview

USB removable media is one of the most common vectors for data exfiltration and malware introduction in corporate environments. Out-of-the-box Ubuntu Linux auto-mounts USB drives silently — any employee can copy sensitive files or introduce malicious payloads with zero authentication required.

This project implements a USB drive access control policy across Ubuntu workstations. The solution intercepts every USB mount request at the OS level using Polkit (PolicyKit) and udisks2, requiring an administrator password before any USB storage device can be accessed. Auto-mounting is simultaneously disabled at the GNOME desktop layer via dconf.

This directly addresses GDPR Article 32, which requires organisations to implement appropriate technical measures to protect personal data against unauthorised access and exfiltration.

THE PROBLEM	OUR SOLUTION
Any employee could insert a USB drive and Ubuntu would auto-mount it silently — copying sensitive files or introducing malware with zero authentication.	Every USB mount request is intercepted by a Polkit rule. Administrator credentials are required before any USB storage device can be accessed. Auto-mounting is disabled at the GNOME dconf layer.

2. Technical Architecture

The solution operates at three layers of the Ubuntu stack:

Component	Role	Our Configuration
Polkit (PolicyKit)	OS-level privilege control for system operations	Rules file or .pkla policy requiring auth_admin before any USB mount
udisks2	Disk management daemon — handles USB mount/unmount events	Actions intercepted: filesystem-mount & filesystem-mount-system
dconf / GNOME	Desktop environment configuration layer	automount=false, automount-open=false (system-wide lock, overrides user settings)

End-to-End Flow

When a USB drive is inserted, the following sequence executes:

- udev detects the USB hardware insertion event
- udisks2 receives a mount request from GNOME Files (Nautilus)
- Mount request is evaluated against the active Polkit policy rules
- Our rule returns AUTH_ADMIN — an administrator password dialog appears
- Correct password entered → drive mounts and becomes accessible
- Password cancelled or incorrect → drive stays unmounted, data inaccessible

3. Version Detection & Script Selection

The correct deployment script depends entirely on the Polkit version installed. The two scripts are not interchangeable — using the wrong one results in silent failure.

Version Check	Polkit Generation	Script to Deploy
pkaction --version → 0.105	Older / Legacy	usb_blocker_for_olderpolkitversions.sh (.pkla format)
pkaction --version → 0.106+	Newer / Modern	usb_blocker_for_newerpolkitversions.sh (.rules JS format)

Run the version check command before proceeding:

```
pkaction --version
```

4. Script A — Newer Polkit (v0.106+)

Deploys a JavaScript-based .rules file — the modern Polkit mechanism. File deployed to:
/etc/polkit-1/rules.d/49-usb-auth.rules

4.1 Execution Steps

- **Step 1 — Make executable:** `chmod +x usb_blocker_for_newerpolkitversions.sh`
- **Step 2 — Run with sudo:** `sudo bash usb_blocker_for_newerpolkitversions.sh`
- **Step 3 — Reboot:** `sudo reboot`
- **Step 4 — Verify:** Insert USB → authentication dialog should appear

4.2 Full Script

```
#!/bin/bash
echo "[+] Configuring USB authentication policy..."

# Create dconf configuration directories
sudo mkdir -p /etc/dconf/db/local.d
sudo mkdir -p /etc/dconf/profile

# Disable GNOME automount
sudo tee /etc/dconf/db/local.d/00-media-automount > /dev/null << 'EOF'
[org/gnome/desktop/media-handling]
automount=false
automount-open=false
EOF

# Create dconf user profile
sudo tee /etc/dconf/profile/user > /dev/null << 'EOF'
user-db:user
system-db:local
EOF
sudo dconf update

# Create Polkit rules directory
sudo mkdir -p /etc/polkit-1/rules.d

# Write authentication rule (core of the policy)
sudo tee /etc/polkit-1/rules.d/49-usb-auth.rules > /dev/null << 'EOF'
polkit.addRule(function(action, subject) {
    if (
        action.id == "org.freedesktop.udisks2.filesystem-mount" ||
        action.id == "org.freedesktop.udisks2.filesystem-mount-system"
    ) {
        return polkit.Result.AUTH_ADMIN;
    }
});
EOF

# Restart Polkit to load rule immediately
sudo systemctl restart polkit 2>/dev/null || true
echo "[+] Done. Run: sudo reboot"
```

4.3 Key Design Decisions

- **AUTH_ADMIN:** Forces administrator (sudo-level) password — not just any user password
- **File numbered 49:** Evaluated before default rules (50+), ensuring our policy takes precedence
- **Both mount actions covered:** filesystem-mount (user-initiated) + filesystem-mount-system (system/removable device)

- **|| true fallback:** Script exits cleanly even if polkit restart fails on unusual systems

5. Script B — Legacy Polkit (v0.105)

Deploys a legacy .pkla INI-format policy file. Strict file ownership (root:root, 644) is mandatory — Polkit 0.105 silently ignores policies with incorrect permissions. File deployed to:
/etc/polkit-1/localauthority/50-local.d/49-usb-auth.pkla

5.1 Execution Steps

- **Step 1 — Make executable:** `chmod +x usb_blocker_for_olderpolkitversions.sh`
- **Step 2 — Run as root:** `sudo bash usb_blocker_for_olderpolkitversions.sh`
- **Step 3 — Reboot:** `sudo reboot`

5.2 Full Script

```
#!/bin/bash
# Must run as root – check EUID
if [ "$EUID" -ne 0 ]; then
    echo "Please run as root (use sudo)."
```

5.3 Key Differences from Script A

- **EUID check:** Script must be invoked as root — no internal sudo calls
- **.pkla format:** INI-style file used exclusively by Polkit 0.105 — incompatible with 0.106+
- **All three Result values:** ResultAny, ResultInactive, ResultActive all set to auth_admin — no session state can bypass
- **Mandatory permissions:** root:root ownership + 644 mode required — Polkit 0.105 silently ignores files with wrong permissions

6. Verification & Testing

6.1 Functional Test

- Insert a USB storage drive into the Ubuntu system
- Wait 3–5 seconds — system should NOT auto-open the drive
- Open Files (Nautilus) and attempt to access the USB drive
- Authentication dialog should appear requesting admin password
- Enter correct admin password — drive mounts and becomes accessible
- Re-insert the drive and cancel the prompt — drive must remain inaccessible

6.2 Verification Commands

For Newer Polkit (0.106+):

```
cat /etc/polkit-1/rules.d/49-usb-auth.rules
cat /etc/dconf/db/local.d/00-media-automount
sudo systemctl status polkit
```

For Legacy Polkit (0.105):

```
cat /etc/polkit-1/localauthority/50-local.d/49-usb-auth.pkla
cat /etc/dconf/db/local.d/00-media-automount
sudo systemctl status polkit
```

7. GDPR Article 32 Alignment

GDPR Article 32 requires organisations to implement appropriate technical measures to ensure a level of security appropriate to the risk — including protection against unauthorised access and exfiltration of personal data. This implementation directly addresses that requirement at the endpoint level.

GDPR Requirement	How This Implementation Addresses It
Prevent unauthorised data transfer	Any USB mount requires explicit admin authentication — prevents silent exfiltration of personal data

Admin accountability	All USB access requires traceable administrator credentials — creates an implicit access audit trail
Data minimisation support	Only authorised personnel can transfer data via removable media
Layered / defence-in-depth	Complements existing network-level controls (firewall, IDS) with endpoint hardening

8. Rollback Procedure

The policy can be safely removed to restore default USB behaviour:

Newer Polkit (0.106+):

```
sudo rm /etc/polkit-1/rules.d/49-usb-auth.rules
sudo rm /etc/dconf/db/local.d/00-media-automount
sudo dconf update && sudo systemctl restart polkit
sudo reboot
```

Legacy Polkit (0.105):

```
sudo rm /etc/polkit-1/localauthority/50-local.d/49-usb-auth.pkla
sudo rm /etc/dconf/db/local.d/00-media-automount
sudo dconf update && sudo systemctl restart polkit
sudo reboot
```

9. Troubleshooting

Issue	Resolution
USB still mounts without password	Confirm Polkit version matches the script used. Run <code>pkaction --version</code> and re-check.
Script exits: 'Please run as root'	Older script requires <code>sudo</code> : <code>sudo bash usb_blocker_for_olderpolkitversions.sh</code>
polkit.service fails to restart	Ignore — settings file already written. Reboot will apply the policy.
KDE or XFCE — policy not working	These DEs have their own automount settings. Disable in System Settings > Removable Devices.
Auth prompt always fails	Ensure you are entering the admin (sudo) user password, not the login user's password.
dconf update command not found	Install: <code>sudo apt install dconf-cli</code> then re-run the script.

10. Summary

Policy Goal	Require admin password for all USB drive mounts on Ubuntu
Method	Polkit (PolicyKit) rules + udisks2 action interception + dconf automount disable
Scripts Delivered	Two scripts — one for Polkit 0.106+ (.rules), one for Polkit 0.105 (.pkla legacy)
Version Check	Run pkaction --version before choosing which script to execute
Tested On	Ubuntu Linux with GNOME desktop environment — live production systems
Compliance	GDPR Article 32 — Technical Security Measures
Status	Successfully implemented and verified
Completed By	Karthikeyan T

#CyberSecurity #EndpointSecurity #Linux #Ubuntu #GDPR #DataLossPrevention #Polkit